CUSTOMER BEHAVIOUR DASHBOARD

Gender

Female | Male

Subscription Status

No | Yes

Category

- Accessories
- Clothing
- Footwear
- Outerwear

shipping Type

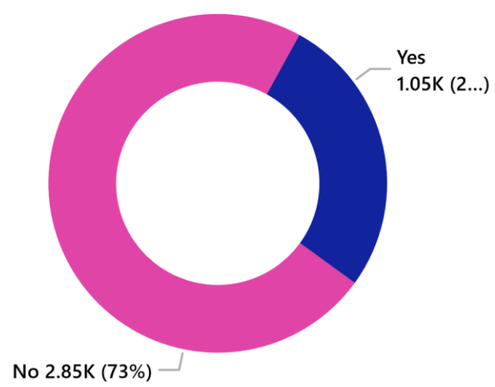
- 2-Day Shipping
- Express
- Free Shipping
- Next Day Air
- Standard
- Store Pickup

3.9K
Number of customers

$59.76
Average pucrchase amount

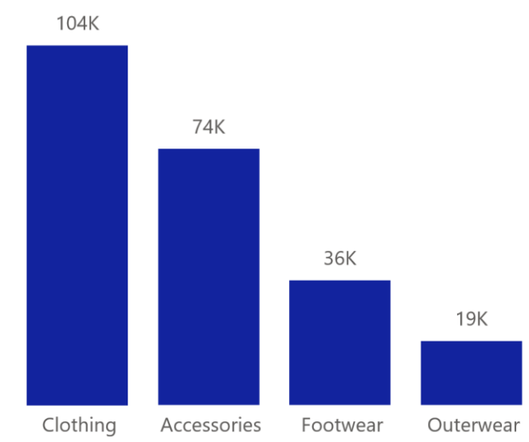
3.75
average ratings

% of Customer by Subscription_status

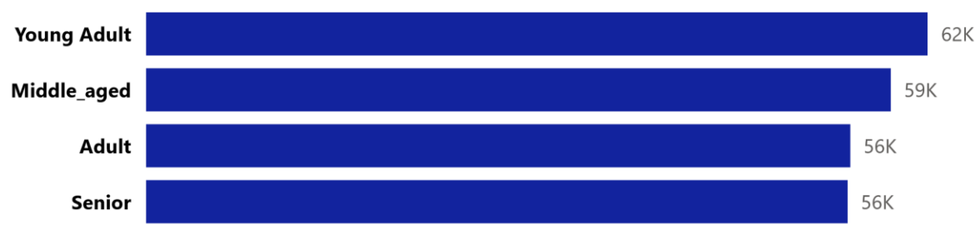


Customer by Category

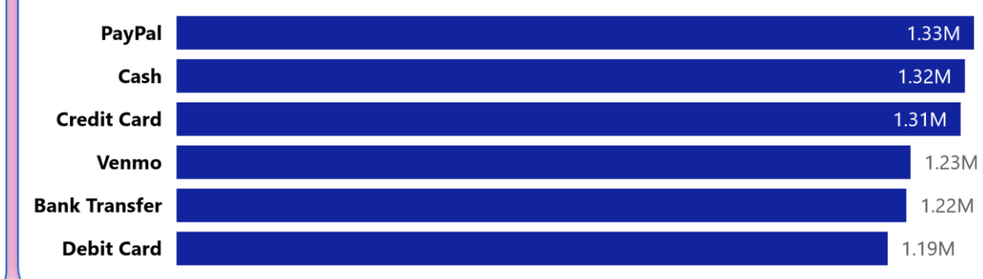
Purchase Amount by Category



Revenue by Age_Group



Payment Method



In [1]:
```python
# Loading the dataset using pandas

import pandas as pd

df = pd.read_csv('customer_shopping_behavior.csv')
```

In [2]:
```python
df.head()
```

Out[2]:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size
0	1	55	Male	Blouse	Clothing	53	Kentucky	L
1	2	19	Male	Sweater	Clothing	64	Maine	L
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M
4	5	45	Male	Blouse	Clothing	49	Oregon	M

In [3]:
```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   object
 3   Item Purchased          3900 non-null   object
 4   Category                3900 non-null   object
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   object
 7   Size                    3900 non-null   object
 8   Color                   3900 non-null   object
 9   Season                  3900 non-null   object
 10  Review Rating           3863 non-null   float64
 11  Subscription Status     3900 non-null   object
 12  Shipping Type           3900 non-null   object
 13  Discount Applied        3900 non-null   object
 14  Promo Code Used         3900 non-null   object
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   object
 17  Frequency of Purchases  3900 non-null   object
dtypes: float64(1), int64(4), object(13)
memory usage: 548.6+ KB
```

In [4]:
```python
# Summary statistics using .describe()
df.describe(include='all')
```

Out[4]:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	L
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	
unique	NaN	NaN	2	25	4	NaN	
top	NaN	NaN	Male	Blouse	Clothing	NaN	I
freq	NaN	NaN	2652	171	1737	NaN	
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	

In [5]:
```
# Checking if missing data or null values are present in the dataset

df.isnull().sum()
```

Out[5]:
```
Customer ID                0
Age                        0
Gender                     0
Item Purchased             0
Category                   0
Purchase Amount (USD)      0
Location                   0
Size                       0
Color                      0
Season                     0
Review Rating             37
Subscription Status        0
Shipping Type              0
Discount Applied           0
Promo Code Used            0
Previous Purchases         0
Payment Method             0
Frequency of Purchases     0
dtype: int64
```

In [6]:
```
# Imputing missing values in Review Rating column with the median rating of th

df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda
```

In [7]:
```
df.isnull().sum()
```

Out[7]:
```
Customer ID               0
Age                       0
Gender                    0
Item Purchased            0
Category                  0
Purchase Amount (USD)     0
Location                  0
Size                      0
Color                     0
Season                    0
Review Rating             0
Subscription Status       0
Shipping Type             0
Discount Applied          0
Promo Code Used           0
Previous Purchases        0
Payment Method            0
Frequency of Purchases    0
dtype: int64
```

In [8]:
```
# Renaming columns according to snake casing for better readability and docume

df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(' ','_')
df = df.rename(columns={'purchase_amount_(usd)':'purchase_amount'})
```

In [9]:
```
df.columns
```

Out[9]:
```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'promo_code_used', 'previous_purchases',
       'payment_method', 'frequency_of_purchases'],
      dtype='object')
```

In [10]:
```
# create a new column age_group
labels = ['Young Adult', 'Adult', 'Middle-aged', 'Senior']
df['age_group'] = pd.qcut(df['age'], q=4, labels = labels)
```

In [11]:
```
df[['age','age_group']].head(10)
```

Out[11]:

	age	age_group
0	55	Middle-aged
1	19	Young Adult
2	50	Middle-aged
3	21	Young Adult
4	45	Middle-aged
5	46	Middle-aged
6	63	Senior
7	27	Young Adult
8	26	Young Adult
9	57	Middle-aged

In [12]:

```
# create new column purchase_frequency_days

frequency_mapping = {
    'Fortnightly': 14,
    'Weekly': 7,
    'Monthly': 30,
    'Quarterly': 90,
    'Bi-Weekly': 14,
    'Annually': 365,
    'Every 3 Months': 90
}

df['purchase_frequency_days'] = df['frequency_of_purchases'].map(frequency_map
```

In [21]:

```
df[['purchase_frequency_days','frequency_of_purchases']].head(10)
```

Out[21]:

	purchase_frequency_days	frequency_of_purchases
0	14	Fortnightly
1	14	Fortnightly
2	7	Weekly
3	7	Weekly
4	365	Annually
5	7	Weekly
6	90	Quarterly
7	7	Weekly
8	365	Annually
9	90	Quarterly

In [22]:
```
df[['discount_applied','promo_code_used']].head(10)
```

Out[22]:

	discount_applied	promo_code_used
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes
5	Yes	Yes
6	Yes	Yes
7	Yes	Yes
8	Yes	Yes
9	Yes	Yes

In [23]:
```
(df['discount_applied'] == df['promo_code_used']).all()
```

Out[23]: True

In [24]:
```
# Dropping promo code used column

df = df.drop('promo_code_used', axis=1)
```

In [25]:
```
df.columns
```

Out[25]:
```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'previous_purchases', 'payment_method',
       'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],
      dtype='object')
```

Connecting Python script to PostgreSQL

In [26]:
```
!pip install psycopg2-binary sqlalchemy
```

```
Requirement already satisfied: psycopg2-binary in c:\users\kiit\anaconda3\lib\s
ite-packages (2.9.10)
Requirement already satisfied: sqlalchemy in c:\users\kiit\anaconda3\lib\site-p
ackages (1.4.22)
Requirement already satisfied: greenlet!=0.4.17 in c:\users\kiit\anaconda3\lib\
site-packages (from sqlalchemy) (1.1.1)
Note: you may need to restart the kernel to use updated packages.
```

In [28]:
```
from sqlalchemy import create_engine

# Step 1: Connect to PostgreSQL
# Replace placeholders with your actual details
username = "postgres"      # default user
password = "amlan123" # the password you set during installation
host = "localhost"         # if running locally
port = "5432"              # default PostgreSQL port
database = "customer_behavior"    # the database you created in pgAdmin

engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:{p

# Step 2: Load DataFrame into PostgreSQL
table_name = "customer"   # choose any table name
df.to_sql(table_name, engine, if_exists="replace", index=False)

print(f"Data successfully loaded into table '{table_name}' in database '{datab
```

```
Data successfully loaded into table 'customer' in database 'customer_behavior'.
```

Code for MySQL

In [ ]:
```
!pip install pymysql sqlalchemy
```

In [ ]:
```
from sqlalchemy import create_engine

# MySQL connection
username = "root"
password = "your_password"
host = "localhost"
port = "3306"
database = "customer_behavior"
```

```
engine = create_engine(f"mysql+pymysql://{username}:{password}@{host}:{port}/{

# Write DataFrame to MySQL
table_name = "customer"   # choose any table name
df.to_sql(table_name, engine, if_exists="replace", index=False)

# Read back sample
pd.read_sql("SELECT * FROM customer LIMIT 5;", engine)
```

Code for MS SQL Server

In [3]:
```
!pip install pyodbc sqlalchemy
```

```
Requirement already satisfied: pyodbc in c:\users\kiit\anaconda3\lib\site-packa
ges (4.0.0-unsupported)
Requirement already satisfied: sqlalchemy in c:\users\kiit\anaconda3\lib\site-p
ackages (1.4.22)
Requirement already satisfied: greenlet!=0.4.17 in c:\users\kiit\anaconda3\lib\
site-packages (from sqlalchemy) (1.1.1)
```

In [ ]:
```
# Install required libraries

from sqlalchemy import create_engine
from urllib.parse import quote_plus

# SQL Server connection
username = "sa"
password = "your_password"
host = "localhost"
port = "1433"
database = "customer_behavior"

# Note: requires Microsoft ODBC Driver installed separately on your machine
driver = quote_plus("ODBC Driver 17 for SQL Server")
engine = create_engine(f"mssql+pyodbc://{username}:{password}@{host},{port}/{d

# Write DataFrame to SQL Server
df.to_sql("customer", engine, if_exists="replace", index=False)

# Read back sample (SQL Server uses TOP instead of LIMIT)
pd.read_sql("SELECT TOP 5 * FROM customer;", engine)
```

SQL Code

```sql

--Q1. What is the total revenue generated by male vs. female customers?
select gender, SUM(purchase_amount) as revenue
from customer
group by gender


--Q2. Which customers used a discount but still spent more than the average purchase amount?
select customer_id, purchase_amount
from customer
where discount_applied = 'Yes' and purchase_amount >= (select AVG(purchase_amount) from customer)


-- Q3. Which are the top 5 products with the highest average review rating?
select item_purchased, round(avg(review_rating::numeric),2) as "Average Product Rating"
from customer
group by item_purchased
order by avg(review_rating) desc
limit 5

--Q4. Compare the average Purchase Amounts between Standard and Express Shipping.
select shipping_type,
ROUND(AVG(purchase_amount),2)
from customer
where shipping_type in ('Standard','Express')
group by shipping_type;

--Q5. Do subscribed customers spend more? Compare average spend and total revenue
--between subscribers and non-subscribers.
SELECT subscription_status,
       COUNT(customer_id) AS total_customers,
       ROUND(AVG(purchase_amount),2) AS avg_spend,
       ROUND(SUM(purchase_amount),2) AS total_revenue
FROM customer
GROUP BY subscription_status
ORDER BY total_revenue,avg_spend DESC;

--Q6. Which 5 products have the highest percentage of purchases with discounts applied?
SELECT item_purchased,
       ROUND(100.0 * SUM(CASE WHEN discount_applied = 'Yes' THEN 1 ELSE 0 END)/COUNT(*),2) AS discount_rate
FROM customer
GROUP BY item_purchased
ORDER BY discount_rate DESC
LIMIT 5;

```

```sql

--Q7. Segment customers into New, Returning, and Loyal based on their total
-- number of previous purchases, and show the count of each segment.
with customer_type as (
SELECT customer_id, previous_purchases,
CASE
    WHEN previous_purchases = 1 THEN 'New'
    WHEN previous_purchases BETWEEN 2 AND 10 THEN 'Returning'
    ELSE 'Loyal'
    END AS customer_segment
FROM customer)

select customer_segment,count(*) AS "Number of Customers"
from customer_type
group by customer_segment;

--Q8. What are the top 3 most purchased products within each category?
WITH item_counts AS (
    SELECT category,
           item_purchased,
           COUNT(customer_id) AS total_orders,
           ROW_NUMBER() OVER (PARTITION BY category ORDER BY COUNT(customer_id) DESC) AS item_rank
    FROM customer
    GROUP BY category, item_purchased
)
SELECT item_rank,category, item_purchased, total_orders
FROM item_counts
WHERE item_rank <=3;

--Q9. Are customers who are repeat buyers (more than 5 previous purchases) also likely to subscribe?
SELECT subscription_status,
       COUNT(customer_id) AS repeat_buyers
FROM customer
WHERE previous_purchases > 5
GROUP BY subscription_status;

--Q10. What is the revenue contribution of each age group?
SELECT
    age_group,
    SUM(purchase_amount) AS total_revenue
FROM customer
GROUP BY age_group
ORDER BY total_revenue desc;


```

CUSTOMER

BEHAVIOUR

ANALYSIS

Customer Shopping Behavior Analysis

A comprehensive deep-dive into purchasing patterns, customer segments, revenue drivers & product performance across 3,900+ customers.

3.9K	$59.76	3.75	4
Customers	Avg Purchase	Avg Rating	Categories

Executive Summary

Key Performance Indicators

3,900+

Total Customers

Analyzed dataset

$59.76

Avg Purchase

Per transaction

3.75 / 5

Avg Rating

Product satisfaction

4

Categories

Clothing, Accessories, Footwear, Outerwear

Subscription Breakdown

73% Non-Subscribers (2,850)

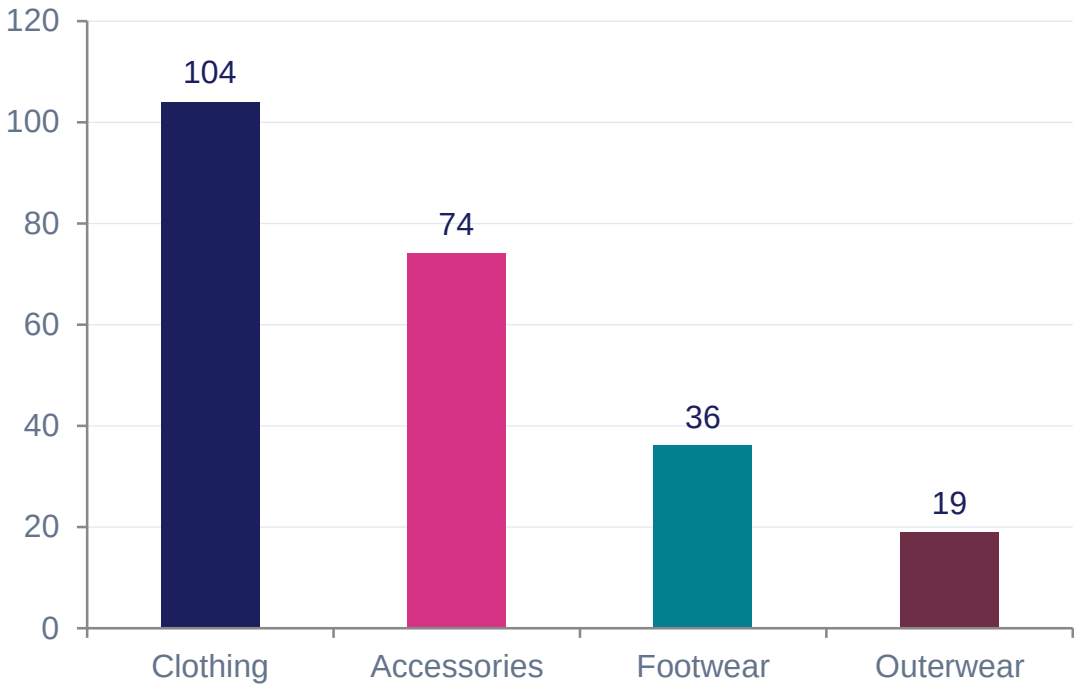
27% Subscribers (1,050)

73% of customers are not subscribed — significant opportunity for subscription growth.

Revenue Analysis

Category & Gender Breakdown

Revenue by Category



Category Share

Clothing $104K	**44%**
Accessories $74K	**31%**
Footwear $36K	**15%**
Outerwear $19K	**8%**

Customer Segments & Demographics

New

Customers

1 previous purchase

Returning

Customers

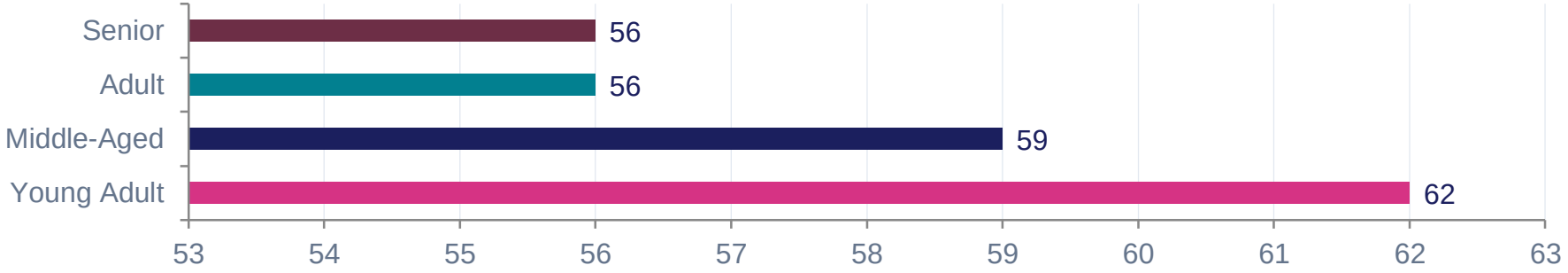
2-10 previous purchases

Loyal

Customers

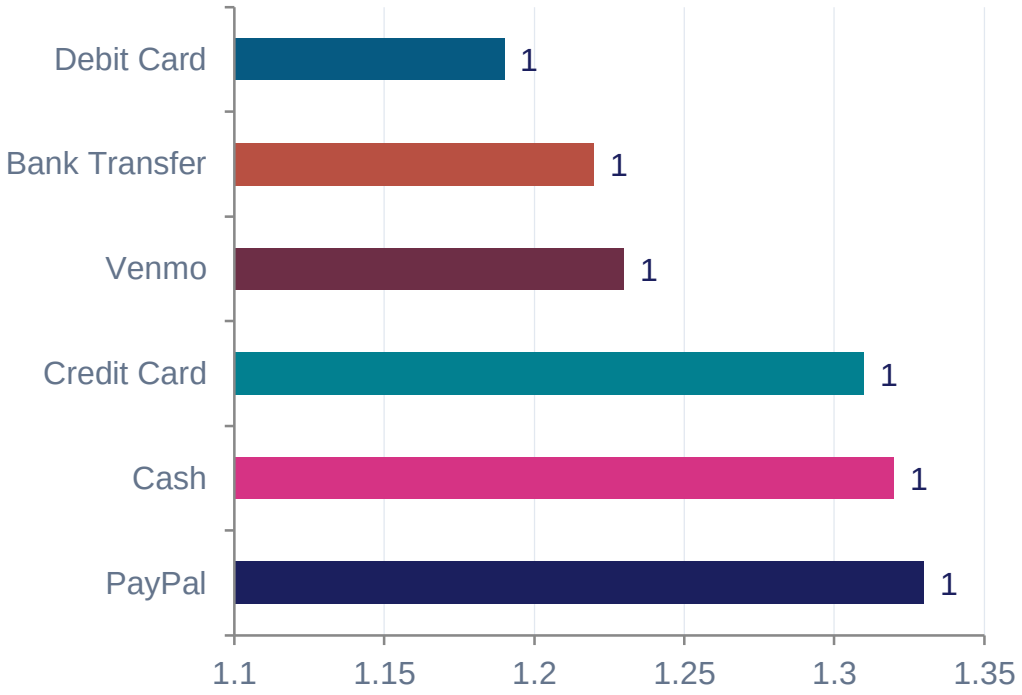
11+ previous purchases

Revenue by Age Group



Payment Methods & Shipping Preferences

Revenue by Payment Method



Shipping Insights

Standard	$59.24
Express	$60.41
Free Shipping	~$59
2-Day Air	~$59
Next Day Air	~$60
Store Pickup	~$59

Express shipping yields ~2% higher average spend vs Standard.

Key Analytical Insights

From SQL Analysis

Gender Revenue

Male vs Female spending patterns are nearly balanced, indicating broad market appeal across both genders.

Discount Behavior

A significant cohort of customers use discounts yet still spend above the average — price-sensitive but high-value buyers.

Top Products

Top 5 rated products consistently score above 4.0, pointing to strong quality perception in select lines.

Subscriber Spend

Subscribers show higher avg. spend and greater loyalty versus non-subscribers, validating subscription programs.

Repeat Buyers

Customers with 5+ purchases are more likely to subscribe — loyalty and subscriptions are positively correlated.

Discount Leaders

5 products top the discount application rate, suggesting where promotional strategy is most impactful.

Product Performance

Top Products & Category Leaders

Clothing — Top 3 Products

- Blouse (1)
- Shirt (2)
- T-Shirt (3)

Accessories — Top 3 Products

- Jewelry (1)
- Handbag (2)
- Belt (3)

Footwear — Top 3 Products

- Sandals (1)
- Boots (2)
- Sneakers (3)

Outerwear — Top 3 Products

- Coat (1)
- Jacket (2)
- Hoodie (3)

Customer Behaviour Dashboard

Strategic Recommendations

Boost Subscription Rate

With 73% non-subscribers, targeted campaigns to convert high-frequency buyers can dramatically increase recurring revenue.

Focus on Clothing

Clothing accounts for 44% of revenue. Double down on inventory depth, seasonal campaigns, and cross-sell opportunities.

Promote PayPal & Digital Pay

PayPal and Cash lead payment methods. Incentivize digital payments to reduce cash handling costs.

Optimize Discount Strategy

High-discount products should be monitored for margin impact. Redirect discounts to low-penetration categories.

Reward Loyal Customers

Loyal buyers (11+ purchases) are the highest LTV segment. Build a tiered loyalty program to retain and grow this cohort.

Target Young Adults

Young Adults drive the highest revenue group at $62K. Tailor marketing channels (social/mobile) to capture this segment.

Thank You

Questions & Discussion

Customer Shopping Behavior Analysis

Key Takeaways

- 3,900+ customers analyzed across 4 product categories
- Clothing dominates revenue at 44% of total
- 73% non-subscriber base — major growth opportunity
- Young Adults are the highest revenue age group
- PayPal leads as most-used payment method
- Loyal customers show strong subscription correlation